

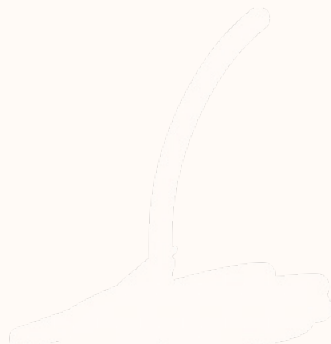
Kiali

Tecnología que cuida sin que lo notes. notes.

Reporte Técnico de Arquitectura y Diseño de Software — Swift Changemakers Hackathon 2026 · iOS Dev Lab · Facultad de Ingeniería, UNAM
Ingeniería, UNAM

Profesor: Ing. Marduk Pérez de Lara Domínguez · Equipo: Swifties · Sánchez Medina José Santiago · Torres Pimentel Obed

Repositorio: github.com/obeedt/AppSwift · Versión del documento: **1.0** · Fecha: **29 de mayo de 2026**



Requerimientos de la Aplicación

Reglas de Negocio

- RN-01: Offline en Rutinas y Perfil
- RN-02: Sin transmisión externa sin consentimiento
- RN-03: Sin autenticación; familiar configura
- RN-04: Rutinas eliminadas no recuperables
- RN-05: Fecha específica o recurrente, no ambas
- RN-06: Máx. 1 tarjeta de intervención activa
- RN-07: Alerta automática por rutina crítica
- RN-08: Mín. 44×44 pt; WCAG AA 4.5:1
- RN-09: Sin sonido/vibración sin habilitación
- RN-10: MVP en memoria; SwiftData bloqueante

Requerimientos Funcionales

Asistente (Must): Saludo contextual, 8 tareas con SF Symbols, consulta libre, guía de pasos, intervención proactiva, confirmación «Ya lo hice».

Should: Animación logo, simulación olvido, TTS parcial. **Could:** STT fuera de MVP.

Rutinas (Must): Lista Pendientes/Completadas, chips de categoría, creación/edición, tipos fecha/recurrente, swipe-to-delete. **Should:** Edición con sheet. **Deferred:** Notificaciones locales v2.0.

Perfil (Must): Avatar, ficha médica, Contacto de Alertas, Emergencias 911, Red de Apoyo. **Should:** Controles TTS.

Requerimientos No Funcionales

- RNF-01: Arranque Asistente <2 seg en en iPhone SE 3.ª gen
- RNF-02: Mín. 44×44 pt táctil
- RNF-03: Dynamic Type hasta xxxLarge (parcial)
- RNF-04: iOS 16.0+ · RNF-05: WCAG AA 4.5:1
- RNF-06: Portrait exclusivo · RNF-07: MVVM
- RNF-08: Datos on-device · RNF-09: Español mexicano + i18n
- RNF-10: Bundle ≤50 MB

Viabilidad Técnica

IA Invisible — MVP

Lógica local: **GestorRutinas** evalúa rutinas vencidas no confirmadas y confirmadas y activa `mostrarIntervencion = true`. Sin modelos de lenguaje, sin APIs externas, sin transmisión de datos. simple y completamente viable en Swift.

Core ML — v3.0+

Modelos de clasificación on-device sobre historial local del usuario (.mlmodel vía Create ML). Sin backend ni costos de API. Requiere datos históricos suficientes.

Viable a corto plazo

- **Notificaciones locales:** `UNUserNotificationCenter` + `UNCalendarNotificationTrigger`
- **TTS:** `AVSpeechSynthesizer` es-MX offline desde iOS 16
- **Persistencia:** `SwiftData @Model` en iOS 17+
- **OCR recetas:** `DataScannerViewController` (VisionKit)
- **Llamadas:** URL scheme `tel://` nativo

Limitaciones reales

- IA conversacional tipo GPT-4 requiere APIs externas con latencia, latencia, costos por token y políticas de privacidad para datos médicos
- Apple Intelligence no disponible en español en el hackathon
- Sincronización familiar requiere backend (Firebase viable en v2.0)

Alcance y MVP

1

v1.0 — MVP (Hackathon)

Asistente con saludo, 8 tareas, intervención proactiva. Rutinas con creación, edición, filtrado y filtrado y completado. Perfil con ficha médica y contactos. Datos en memoria.

2

v2.0 — 3 a 4 semanas

SwiftData, notificaciones locales, TTS real, llamadas directas. Costo estimado: 70,000–133,000 133,000 MXN.

3

v3.0 — 8 a 12 semanas

CloudKit, VisionKit (OCR recetas), dashboard familiar, sincronización en tiempo real.

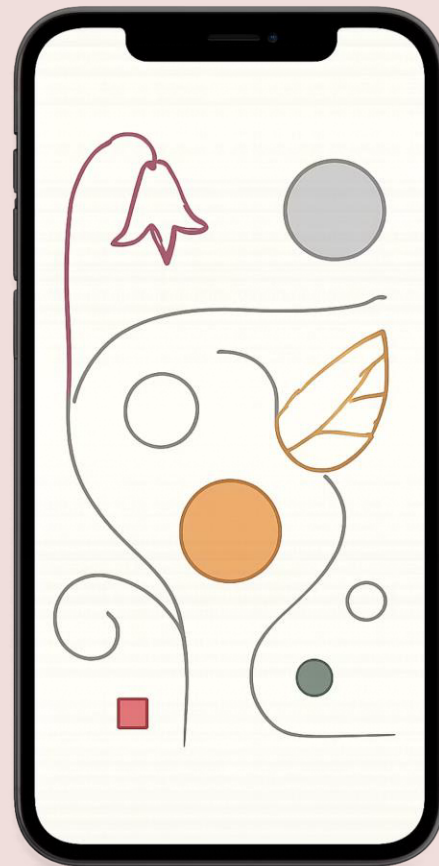
4

v4.0 — 16 a 20 semanas

App Intents para Siri, watchOS companion app, Core ML para personalización de intervenciones. intervenciones.

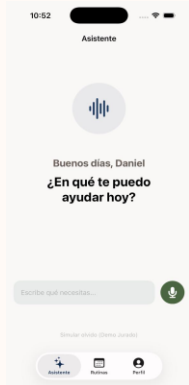


Excluido del MVP: Notificaciones locales, TTS funcional (toggle presente, AVSpeechSynthesizer no AVSpeechSynthesizer no integrado), reconocimiento de voz, persistencia entre sesiones, autenticación y servidores externos.



Wireframes — 10 Pantallas

PANTALLAS 1-5



1. AsistenteView

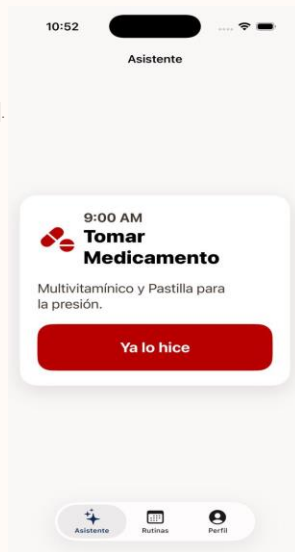
Saludo contextual + cuadrícula 8 tareas tareas frecuentes. Datos estáticos en en AppThemes.swift; vigencia permanente. Estado: @State var consultaActual.

2. TarjetaGuia

Guía de 3-6 pasos en lenguaje llano. Diccionario estático [String: [String]]. Solo lectura; oculta al presionar «Listo».

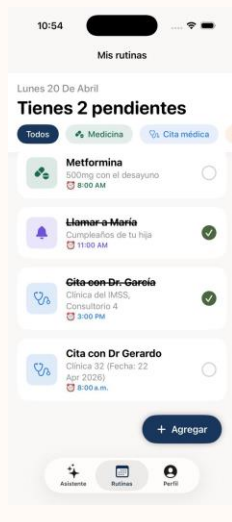
3. TarjetaIntervencion

Overlay proactivo. Botón «Ya lo hice» 56pt (#800000). Fuente: @ObservedObject gestorRutinas. Ejecuta confirmarIntervencion().



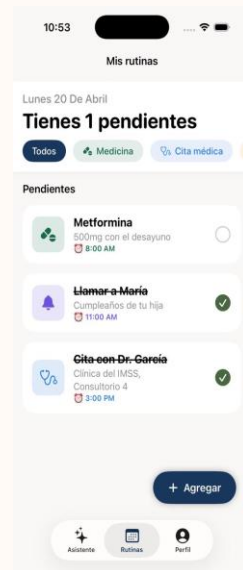
5. EditorRutinaSheet

Sheet modal crear/editar rutina. Toggle fecha específica/recurrente, DatePicker, selector de categoría. Valida título no vacío. Llama a agregarRutina() o actualizarRutina().



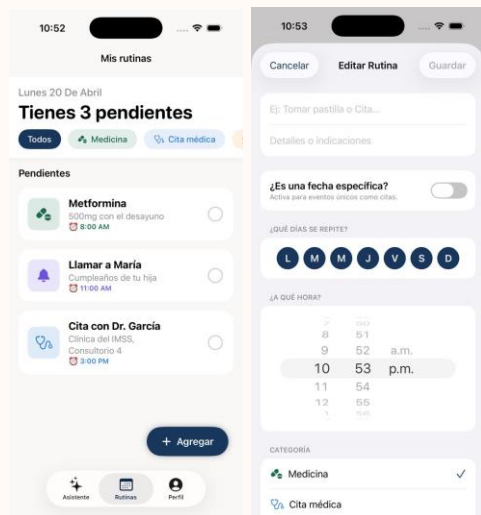
4. RecordatoriosView

Lista Pendientes/Completadas con chips de filtro. Fuente: gestorRutinas.pendientesHoy. Operaciones: toggleCompletada, eliminarRutinas.



Wireframes — Pantallas 6–10

6. Selector Días Recurrentes
 7 chips 44pt (LMMVSD). @State var
 díasSeleccionados: Set<Int>. Guardar deshabilitado si Set vacío.



7. Perfil: Identidad y Ficha Médica

Avatar, nombre, edad, tipo sangre, alergias. Datos estáticos en MVP; SwiftData en v2.0.



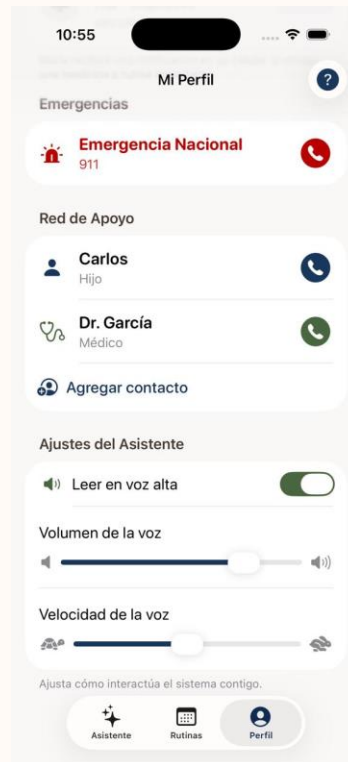
8. Contacto de Alertas y Emergencias

Avatar familiar vinculado, badge «Dispositivo vinculado», botón 911, Red de Apoyo con tel://. Notificación automática si rutina crítica sin confirmar.



9. Ajustes del Asistente

Toggle TTS
 (@AppStorage("leerVozAlta")),
 sliders volumen/velocidad con SF
 Symbols. Persistencia vía UserDefaults.



10. Completadas y Edición

Ítems con tachado y checkmark verde.
 Editor pre-llenado al tap. Botón
 «Eliminar rutina» en rojo. Reactividad
 automática vía @Published.

Flujo de Navegación y Gestos

Arquitectura de Navegación

Capa 1 — TabView persistente: Asistente (índice 0, activo al inicio), Rutinas, Perfil. Tinte azulMarino (#1A3560).

Capa 2 — Sheet modals exclusivos: EditorRutinaSheet para crear/editar rutinas. Sin NavigationStack: los adultos mayores reportan dificultad con el botón Back. Sheets con botones Cancelar/Guardar eliminan ambigüedad del swipe-down.

Flujo típico: Asistente → TarjetaIntervencion → «Ya lo hice» → Rutinas → filtrar → crear recordatorio → Perfil. Máximo 1 nivel de profundidad.

Gestos Especiales

- **Swipe-to-delete** en Rutinas: expone botón rojo estándar iOS (`.onDelete`). Tutorial contextual en v2.0.
- **Tap en chips:** transición 0.2s con `withAnimation`. Área 36×48pt.
- **Tap en ícono de estado:** animación spring para completar rutina.
- **Sheet con `.interactiveDismissDisabled(true)`** al modificar campos: previene pérdida accidental por gestos involuntarios.



Los gestos reutilizan patrones de iOS nativo (WhatsApp, Fotos): el usuario ya los conoce sin aprendizaje explícito.

Flujo de Datos, Estado y Servicios

Single Source of Truth

GestorRutinas (@StateObject + @EnvironmentObject) posee el estado el estado global. Entidad central: `struct Recordatorio` con id, titulo, detalle, hora, categoria, completada, esFechaEspecifica, fechaEspecifica, dias: Set<Int>.

Ciclo de vida: **Activa-Pendiente** → **Intervención** → **Completada**.

Eliminación terminal sin papelera.

Operaciones CRUD por pantalla

- **Rutinas:** Read (pendientesHoy), Create (agregarRutina), Update (toggleCompletada, actualizarRutina), Delete (eliminarRutinas)
- **Intervención:** Update (confirmarIntervencion)
- **Perfil llamadas:** Exec (UIApplication.shared.open tel://)

Almacenamiento Local

MVP: Todos los datos en memoria. Solo `@AppStorage("leerVozAlta")` en UserDefaults.

v2.0 — SwiftData: @Model en Recordatorio, @Query en GestorRutinas, ModelContext inyectado. Cero transmisión externa sin consentimiento (LFPDPPP: datos de salud = sensibles).

Servicios Externos (post-MVP)

- **Firebase Cloud Messaging:** Push al familiar. Plan Spark gratuito gratuito hasta 1M mensajes/mes.
- **CloudKit:** Sincronización familiar. Incluido en Apple Developer Program (\$99 USD/año).
- **HealthKit:** Requiere entitlement especial; revisión App Store: 2–4 semanas.

Equipo, Administración y Estimaciones

Equipo Swifties

Sánchez Medina — Arquitecto/Technical Lead: MVVM, Models.swift, Models.swift, GestorRutinas, módulo Asistente.

Torres Pimentel — UI/UX/Frontend Lead: AppTheme, RecordatoriosView, EditorRutinaSheet, PerfilView, demo.

Metodología

Micro-sprints de 90 min con objetivos de entrega vertical. Checkpoints al 80%; Checkpoints al 80%; reducción de scope ante retrasos. Disciplina contra scope contra scope creep.

Estimaciones de Tiempo

- v2.0: 3–4 semanas
- v3.0: 8–12 semanas
- v4.0: 16–20 semanas

Margen de error: 30–50% hacia arriba.

Costos y Sostenibilidad

v2.0: 70,000–133,000 MXN (2 devs semi-senior). Apple Developer: \$99 \$99 USD/año. Modelo freemium: Kiali Plus \$79 MXN/mes. Meta 1,000 1,000 suscriptores = ~699,000 MXN/año. Apple retiene 30% año 1, 15% año 15% año 2+.



App Store: MVP requiere cero permisos. v2.0: NSUserNotificationsUsageDescription, NSMicrophoneUsageDescription, NSSpeechRecognitionUsageDescription. v3.0: NSCameraUsageDescription. v4.0: entitlement HealthKit. Privacy Policy y Nutrition Label obligatorios.